Artificial Intelligence Assignment: General Purpose AI Reasoning System

Jahanvi Thumar
M26CSA016

October 2026

Abstract

This project introduces an artificial intelligence reasoning system that uses the programming language Python to answer whether a query can be supported by the knowledge base expressed using natural language. In this solution, all facts and rules are converted into internal logical representation, and classical reasoning techniques are employed to draw conclusions. The reasoning process is performed by forward chaining and unification. The task of finding facts, rules, and queries is performed using a pattern-based parser that works with natural language. This approach is evaluated in terms of performance on the RuleTaker, FOLIO, and CounterBench datasets available to the general public.

1 Introduction

The objective of this project is to develop an AI system capable of determining whether a given statement logically follows from a knowledge base containing facts and rules.

The system follows the pipeline:

Natural Language → Internal Representation → Classical Reasoning → Prediction

For example, consider the following knowledge base:

- Bob is young.
- Bob is kind.
- If someone is young, then they are smart.
- If someone is smart and kind, then they are successful.

The query is:

Bob is successful.

The system should derive that Bob is smart and subsequently derive that Bob is successful. Therefore, the answer is True.

The implementation uses Python and classical AI reasoning techniques. The final prediction is generated by the reasoning engine rather than by a generative AI model.

2 System Architecture

The system consists of the following main components:

1. **Dataset Processing:** Loads and prepares the public reasoning datasets.

2. **Natural Language Parser:** Converts supported sentences into logical facts and rules.
3. **Knowledge Representation:** Stores facts and rules using structured logical objects.
4. **Unification:** Matches variables in rules with constants in known facts.
5. **Forward Chaining:** Repeatedly applies rules to derive new facts.
6. **Query Evaluation:** Checks whether the queried fact can be derived.
7. **Evaluation and Submission:** Measures accuracy and inference time and generates predictions in CSV format.

3 Knowledge Representation

The system represents information using two main data structures: literals and rules.

3.1 Literals

A literal consists of:

- Predicate name
- Arguments
- Negation flag

For example, the sentence "Bob is young" is represented as:

```
Literal(
    predicate="Young",
    args=("Bob",),
    negated=False
)
```

A relation such as "Bob likes Alice" can be represented as:

```
Literal(
    predicate="Likes",
    args=("Bob", "Alice"),
    negated=False
)
```

Variables are represented using names such as `?x`.

3.2 Rules

A rule consists of a body and a head.

For example:

$$Young(x) \rightarrow Smart(x)$$

is represented as a rule with:

- Body: $Young(x)$
- Head: $Smart(x)$

A rule with multiple conditions is represented as:

$$Smart(x) \land Kind(x) \rightarrow Successful(x)$$

This representation allows the reasoning engine to match known facts against rule conditions and derive new facts.

4 Natural Language Parsing

The system uses a pattern-based parser to convert supported natural-language statements into literals and rules.

The parser handles patterns including:

- Simple unary facts, such as "Bob is young".
- Binary relations, such as "Bob likes Alice".
- Conditional statements, such as "If someone is young, then they are smart".
- Rules containing multiple conditions.
- Some statements containing pronouns.

The parser separates the context into facts and rules and converts the query into the same internal representation.

The parser is intentionally limited to recognized sentence patterns. Sentences outside these patterns may be marked as unsupported.

5 Reasoning Algorithms

5.1 Unification

Unification is used to match variables in a rule with constants in known facts.

For example, consider:

$$Young(?x)$$

and the known fact:

$$Young(Bob)$$

Unification produces the substitution:

$$\{?x \mapsto Bob\}$$

The substitution is then applied to the rule's conclusion.

The implementation checks that predicates, argument counts, and negation states match, and that variable assignments remain consistent.

5.2 Forward Chaining

Forward chaining is the main inference algorithm.

The algorithm begins with the facts in the knowledge base. It repeatedly checks whether the conditions of any rule match the known facts.

When a rule matches, its conclusion is added to the set of known facts if it is not already present.

The process continues until no new facts can be derived or the maximum number of reasoning rounds is reached.

The main steps are:

1. Initialize the known facts.
2. Select a rule.
3. Match its body against known facts using unification.
4. Apply the resulting substitutions to its head.
5. Add newly derived facts.
6. Repeat until no additional facts are produced.

5.3 Query Evaluation

After forward chaining, the system checks whether the query is present in the derived fact set.

If the query is found, the prediction is True. Otherwise, the prediction is False, provided the input was successfully processed.

If the parser cannot handle the input, the system can mark the example as unsupported rather than treating the parser failure as a logical conclusion.

5.4 Counterfactual Reasoning

An additional deterministic causal-graph solver was implemented for a limited subset of CounterBench.

It supports simple causal statements such as:

$$A \to C$$

$$A \land B \to C$$

$$A \lor B \to C$$

For supported counterfactual questions, the solver intervenes on a variable by setting it to False and evaluates the resulting causal graph.

A topological ordering is used to evaluate the variables. Graphs containing cycles are rejected.

Unintervened root variables are assumed to be True in this limited deterministic model. Multiple sufficient-cause clauses are treated as alternative ways of producing an effect.

This solver supports only a restricted subset of counterfactual language and does not implement general structural causal reasoning.

6 Experimental Evaluation

The system was evaluated using the public RuleTaker, FOLIO, and CounterBench datasets.

Accuracy was calculated as:

$$Accuracy = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Examples}} \times 100$$

Parser coverage was calculated as:

$$Coverage = \frac{\text{Number of Supported Examples}}{\text{Total Number of Examples}} \times 100$$

6.1 RuleTaker

RuleTaker was used to evaluate the main forward-chaining reasoning engine.

The development evaluation contained 300 examples.

Table 1: RuleTaker Results

Metric	Result
Examples evaluated	300
Accuracy	71.67%
Parser coverage	100%
Average inference time per example	0.000728 seconds

The results show that the system can process the evaluated RuleTaker examples and perform logical inference.

However, parser support does not guarantee that every sentence has been interpreted correctly. The error analysis also helps identify incorrect predictions and false negatives.

6.2 FOLIO

FOLIO contains three possible labels: True, False, and Uncertain.

For binary evaluation, examples labelled Uncertain were excluded. This left 677 examples from the original 1001.

Table 2: FOLIO Results

Metric	Result
Total examples	1001
Uncertain examples	324
Binary examples evaluated	677
Parser-supported examples	45
Parser coverage	6.65%
Accuracy on all binary examples	31.31%
Accuracy on supported examples	40.00%
Total inference time	0.373 seconds

The low parser coverage indicates that the current pattern-based parser cannot handle most of the more complex FOLIO statements.

The supported-only accuracy is calculated separately to show performance on examples that the parser could process. It should not be interpreted as the overall FOLIO accuracy.

6.3 CounterBench

CounterBench was loaded and inspected as part of the dataset preparation stage.

The dataset was converted into a Pandas DataFrame by combining the records from the available JSON files. The inspection process displayed the column names, total number of examples, sample records, question types, and answer distribution.

The dataset contained 1200 examples with the following columns:

- question
- given_info
- answer
- type
- question_id
- meta
- source_file

This inspection helped identify the structure of the CounterBench dataset and its available information for potential reasoning and evaluation.

7 Example Reasoning Trace

Consider the following knowledge base:

- Bob is young.
- Bob is kind.
- If someone is young, then they are smart.
- If someone is smart and kind, then they are successful.

Query:

> Bob is successful.

The internal representation contains the facts:

$$Young(Bob)$$

$$Kind(Bob)$$

and the rules:

$$Young(x) \rightarrow Smart(x)$$

$$Smart(x) \land Kind(x) \rightarrow Successful(x)$$

7.1 Inference Round 1

The first rule matches the known fact:

$$Young(Bob)$$

Unification produces:

$$\{x \mapsto Bob\}$$

The system derives:

$$Smart(Bob)$$

7.2 Inference Round 2

The second rule now matches both:

$$Smart(Bob)$$

and:

$$Kind(Bob)$$

The system derives:

$$Successful(Bob)$$

7.3 Final Prediction

Since the query is present in the derived fact set:

$$\boxed{Prediction = True}$$

This trace demonstrates how unification and forward chaining are used together to derive a conclusion from the knowledge base.

8 Limitations

The current implementation has several limitations:

1. The natural-language parser is pattern-based and supports only a limited range of sentence structures.
2. Complex linguistic constructions, quantifiers, and more advanced logical expressions may not be parsed correctly.

3. FOLIO coverage is low because its statements often require more expressive logical representations.
4. The CounterBench causal solver handles only a small subset of deterministic causal structures and question templates.
5. The assumption that unintervened root variables are True limits the causal solver's generality.
6. Unsupported examples reduce the overall coverage and can affect the final prediction accuracy when a fallback label is required.

These limitations show that the main challenge is not only the inference algorithm but also converting natural language into a sufficiently expressive and accurate representation.

9 Conclusion

This technique implemented by the project enables the utilization of classical AI reasoning systems based on Python in order to convert supported facts and natural language rules into the internal representation of the information and apply unification and forward chaining principles for obtaining the results.

The RuleTaker tests show how the major reasoning engine operates, while the FOLIO tests show the limitations of the pattern-based parser as far as complex logical statements are concerned. The limited deterministic causal graph solver gives some opportunities for counterfactual reasoning to CounterBench.

In addition, the system is capable of tracing the reasoning process by providing an explanation of how the new information was obtained.

In general, the project has demonstrated the full classical reasoning pipeline, identifying parser limitations and expressivity as the areas that need further development.

References

[1] P. Clark, O. Tafjord, and K. Richardson, "Transformers as Soft Reasoners over Language," in *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI)*, 2020, pp. 3882–3890. https://doi.org/10.24963/ijcai.2020/537

[2] S. Han et al., "FOLIO: Natural Language Reasoning with First-Order Logic," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 22017–22031. https://doi.org/10.18653/v1/2024.emnlp-main.1229

[3] Y. Chen, V. K. Singh, J. Ma, and R. Tang, "CounterBench: Evaluating and Improving Counterfactual Reasoning in Large Language Models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 36, 2026, pp. 30350–30358. https://doi.org/10.1609/aaai.v40i36.40287

[4] Python Software Foundation, "Python Documentation." https://docs.python.org/3/

[5] The pandas development team, "pandas: Python Data Analysis Library." https://pandas.pydata.org/docs/